LLD – Moderation Module (OpenScholar)

1. Overview

This document defines the Low-Level Design (LLD) for the Moderation module of OpenScholar. The module governs the lifecycle of research papers through administrative review, including approval and rejection, with full audit logging and state transition validation.

---

2. Scope

The Moderation module is responsible for:

- Review submitted papers (status: pending)
- Approve or reject papers
- Maintain moderation logs (audit trail)
- Enforce valid state transitions
- Trigger notifications (via Notification module)

Out of Scope:

- Paper creation and editing (Paper module)
- User notifications delivery (Notification module)

---

3. Data Model (Relevant Tables)

Tables used:

- papers
- paper_versions
- moderation_logs
- users (admin)

papers (fields used)

- id (UUID)
- status (ENUM: draft | pending | approved | rejected)
- created_by (UUID)
- is_deleted (BOOLEAN)

paper_versions (fields used)

- id (UUID)
- paper_id (UUID)

- is_published (BOOLEAN)
- created_at (TIMESTAMP)

moderation_logs

- id (UUID)
- paper_id (UUID)
- admin_id (UUID)
- action (ENUM: approved | rejected)
- reason (TEXT, nullable)
- created_at (TIMESTAMP)

Constraints:

- paper_id references papers.id
- admin_id references users.id

---

4. State Machine (Paper Lifecycle)

Valid transitions:

- draft → pending
- pending → approved
- pending → rejected
- rejected → pending (resubmission via Paper module)

Invalid transitions must be rejected at service layer.

---

5. API Contracts

5.1 Get Pending Papers (Admin)

Endpoint: GET /api/admin/papers/pending

Query Parameters:

- page (default: 1)
- limit (default: 10)

Response: { "total": 25, "results": [{ "paperId": "uuid", "title": "Paper Title", "submittedAt": "timestamp", "author": "User Name" }] }

5.2 Approve Paper

Endpoint: POST /api/admin/papers/{paperId}/approve

Headers: Authorization: Bearer <admin_jwt>

Response: { "message": "Paper approved successfully" }

5.3 Reject Paper

Endpoint: POST /api/admin/papers/{paperId}/reject

Headers: Authorization: Bearer <admin_jwt>

Request Body: { "reason": "Insufficient originality" }

Response: { "message": "Paper rejected" }

5.4 Get Moderation History

Endpoint: GET /api/admin/papers/{paperId}/logs

Response: { "logs": [{ "action": "approved", "admin": "Admin Name", "reason": null, "timestamp": "..." }] }

---

6. Service Logic

6.1 getPendingPapers()

Steps: 1. Validate admin role 2. Query papers where status = 'pending' and is_deleted = false 3. Join latest paper_version for title 4. Join creator (user) 5. Apply pagination 6. Return results

---

6.2 approvePaper(paperId, adminId)

Steps: 1. Validate admin role 2. Fetch paper 3. Ensure current status = 'pending' 4. Update paper.status = 'approved' 5. Set latest paper_version.is_published = true 6. Insert moderation_logs (action = approved) 7. Trigger notification (async)

6.3 rejectPaper(paperId, adminId, reason)

Steps: 1. Validate admin role 2. Fetch paper 3. Ensure current status = 'pending' 4. Update paper.status = 'rejected' 5. Insert moderation_logs (action = rejected, reason) 6. Trigger notification (async)

6.4 getModerationLogs(paperId)

Steps: 1. Validate admin role 2. Query moderation_logs by paper_id 3. Join admin user info 4. Return ordered logs (latest first)

---

7. Validation Rules

- Only admin users can access moderation APIs
- Paper must exist and not be deleted
- Only pending papers can be approved/rejected
- Reject action must include reason (non-empty)

---

8. Security Considerations

- Enforce RBAC at API layer (admin only)
- Validate JWT and role before actions
- Prevent unauthorized access to moderation endpoints

---

9. Consistency & Transactions

- Use database transaction for approve flow:
- update paper status
- update paper_version
- insert moderation log
- Rollback on failure to maintain consistency

---

10. Edge Cases

- Approving already approved paper
- Rejecting already rejected paper
- Concurrent admin actions (race condition)
- Missing latest version

---

11. Prisma Model Mapping

model ModerationLog { id String @id @default(uuid()) paperId String adminId String action String reason String? createdAt DateTime @default(now()) }

---

12. Folder Structure

src/ app/api/admin/papers/pending/route.ts app/api/admin/papers/[id]/approve/route.ts app/api/admin/papers/[id]/reject/route.ts app/api/admin/papers/[id]/logs/route.ts

modules/moderation/service.ts modules/moderation/repository.ts

---

13. Implementation Notes

- Always fetch latest version when approving
- Keep moderation logic isolated from Paper module
- Use transactions for critical updates

---

14. Acceptance Criteria

- Admin can view pending papers
- Admin can approve or reject papers
- System enforces valid state transitions
- Moderation actions are logged
- Notifications are triggered

---

15. Future Enhancements

- Multi-level moderation (reviewers)
- Moderation dashboard analytics
- Bulk approval/rejection
- Automated plagiarism checks integration

End of Document